



ರೈ ತಾಂತ್ರಿಕ ವಿಶ್ವವಿದ್ಯಾಲಯ

Rai Technology University

(Estd. under Karnataka Act. No. 40 of 2013)

ASSIGNMENT-1

Subject: Deep learning

Subject Code: 0CS351T

Submitted By:

Student Name: SUHANA

University Enrolment Number: RTU24101CS010

Program Name: B.TECH CSE

Semester: 5TH Sem

Session: 2026-2027

Submitted To:

Faculty Name: Mr. SATYAM

Department of Computer Science & Engineering College

Of Engineering

Rai Technology University

Mnist Handwritten Digit Classification using Neural Network. Report

1. Aim

To build a simple neural network using Tensorflow keras to classify handwritten digits from 0 to 9 using the Mnist dataset.

2. Introduction

Mnist is a popular dataset used for handwritten digit recognition. It contains grayscale images of handwritten digits from 0 to 9. Each image has a size of 28×28 pixels. The dataset contains 60,000 training images and 10,000 testing images. In this project, a neural network is trained on the training data and then evaluated on unseen test data.

3. Tools and Technologies.

Python, tensorflow, keras, Numpy, Matplotlib, and Google Colab Notebook.

4. Dataset loading and Exploration.

The MNIST dataset was loaded using the Tensorflow keras dataset library. The training images have shape (60000, 28, 28), while the test images have shape (10000, 28, 28), pixel values range from 0 to 255 & were normalized to 0-1 by dividing by 255. Sample images were

2
displayed using Matplotlib to understand the handwritten digits.

5. Neural Network Architecture.

The model uses an input image to 28×28 pixels, a flatten layer, a Dense hidden layer with 128 neurons and ReLU activation, a 20% Dropout layer, and a final Dense layer with 10 neurons and Softmax activation. The ten output neurons represent digits 0 through 9.

6. Model Compilation and training

The model was compiled using the adam optimizer, sparse categorical Crossentropy loss, and accuracy as the evaluation metric. It was trained for 10 epochs with a batch size of 32. Ten percent of the training data was used for validation.

Code

```
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt

Print("Tensorflow version:", tf.__version__)

# Load MNIST dataset.
(x_train, y_train), (x_test, y_test) = tf.keras.datasets.mnist.load_data()

Print("Training images shape:", x_train.shape)
Print("Training labels shape:", y_train.shape)
Print("Testing images shape:", x_test.shape)
Print("Testing labels shape:", y_test.shape)
```

```
Print ("First 10 labels:")
```

```
Print (y_train[:10])
```

```
Print ("\nminimum pixel value:", x_train.min())
```

```
Print ("\nmaximum pixel value:", x_train.max())
```

```
plt.figure(figsize=(10,5))
```

```
for i in range(10):
```

```
plt.subplot(2,5,i+1)
```

```
plt.imshow(x_train[i], cmap='gray')
```

```
plt.title(f"label: {y_train[i]}")
```

```
plt.axis('off')
```

```
plt.tight_layout()
```

```
plt.show()
```

~~##~~ Normalize data.

```
x_train = x_train.astype("float32") / 255.0
```

```
x_test = x_test.astype("float32") / 255.0
```

~~##~~ Create model.

```
model = tf.keras.Sequential([
```

```
tf.keras.layers.Input(shape=(28,28)),
```

```
tf.keras.layers.Flatten(),
```

```
tf.keras.layers.Dense(128, activation='relu'),
```

```
tf.keras.layers.Dropout(0.2),
```

```
tf.keras.layers.Dense(10, activation='softmax')
```

```
])
```

~~##~~ Compile model

```
model.compile
```

```
optimizer='adam',
```

loss = 'sparse - categorical - crossentropy',
metrics = ['accuracy']
)

model.summary()

train model

history = model.fit(
 x=train,
 y=train,
 epochs=10,
 batch_size=32,
 validation_split=0.1
)

Evaluate model

test_loss, test_accuracy = model.evaluate(
 x=test,
 y=test,
 verbose=0
)

print("\n Test loss:", test_loss)

print(" Test Accuracy:", test_accuracy)

print(" Test Accuracy (%):", test_accuracy * 100)

Accuracy graph.

plt.figure(figsize=(8,5))

plt.plot(history.history['accuracy'], label='training Accuracy')

plt.plot(history.history['val_accuracy'], label='validation Accuracy')

plt.title("training and validation Accuracy")

plt.xlabel("Epoch")

plt.ylabel("Accuracy")

```
plt.legend()
plt.grid(True)
plt.show()
```

loss graph

```
plt.figure(figsize=(8,5))
plt.plot(history.history['loss'], label='training loss')
plt.plot(history.history['val_loss'], label='validation loss')
plt.title("training and validation loss")
plt.xlabel("Epoch")
plt.ylabel("loss")
plt.legend()
plt.grid(True)
plt.show()
```

Predict 5 images

```
sample_images = x_test[:5]
```

```
actual_labels = y_test[:5]
```

```
Predictions = model.predict(sample_images, verbose=0)
```

```
Predicted_labels = np.argmax(predictions, axis=1)
```

```
Print("In actual vs predicted")
```

```
for i in range(5):
```

```
    print (
```

```
        f"Image {i+1}:"
```

```
        f"Actual = {actual_labels[i]},"
```

```
        f"predicted = {predicted_labels[i]}"
```

```
    )
```

Display prediction.

```
Pred = model.predict(x_test[:5], verbose=0)
```

```
for i in range(5):
```

```
    print("Actual:", y_test[i],
```

```
          "predicted:", np.argmax(pred[i]))
```

```
# Experiment: 64 neurons.
```

```
model2 = tf.keras.Sequential([
```

```
    tf.keras.layers.Flatten(),
```

```
    tf.keras.layers.Dense(64, activation='relu'),
```

```
    tf.keras.layers.Dense(10, activation='softmax')
```

```
])
```

```
model2.compile(optimizer='adam',
```

```
               loss='sparse_categorical_crossentropy',
```

```
               metrics=['accuracy'])
```

```
model2.fit(x_train, y_train, epoch=5, verbose=0)
```

```
loss, acc = model2.evaluate(x_test, y_test, verbose=0)
```

```
print("original accuracy:", test_accuracy)
```

```
print("Experiment accuracy:", acc).
```

Model Comparison.

original: 97.98%

64 neuron: 97.41%

Conclusion

The neural network successfully classified MNIST handwritten digits with good accuracy.